

- It should be noted that the equality test between any two ordered dictionaries, is order sensitive, while the comparison between an ordered dictionary and another mapping object like a `dict` is order insensitive. If we have two ordered dictionaries with identical elements but different element ordering, on applying an equality operator we will get a `False` value.

```
>>> od1 = OrderedDict({'a': 1, 'b': 2, 'c': 3})
>>> od2 = OrderedDict({'c': 3, 'b': 2, 'a': 1})
>>> od1 == od2
False
```

In comparison with another regular dictionary, which contains the same elements in an arbitrary order; we will get a `True` equality. Hence `OrderedDict` can be substituted anywhere a `dict` is used.

```
>>> d1 = dict({'c': 3, 'b': 2, 'a': 1})
>>> d1 == od1
True
```

namedtuple

`namedtuple` is a container that is very similar to a `dict` in a way that both contain keys that are hashed to a particular value, but it allows index-based access to the fields, just like a conventional `tuple`.

A `namedtuple` allows multiple variations of access operations. Let's illustrate the same using the following example:

```
>>> Employee = namedtuple('Employee', ('name', 'age', 'address'))
>>> empl = Employee('David', '10', 'Mumbai')
```

- Using index of the value:

```
>>> empl[0]
'David'
```

- Using the `name` or the `key` of the corresponding value:

```
>>> empl.age
'10'
```

- Using `getattr()`, which is a built-in function in the python library. The function accepts the `namedtuple` object and the `key` as parameters.

```
>>> getattr(empl, 'address');
'Mumbai'
```

A `namedtuple` also provides numerous methods that enable conversion